

Build & Run Instructions

This document provides all necessary instructions to compile and run the **Smart Cabin & Vehicle Health Monitoring System**. The project is built using modern C++17 and relies on standard libraries.

Prerequisites

Before you begin, ensure you have the following installed on your system:

Tool	Minimum Version	Purpose
C++ Compiler	C++17 support	GCC $\geq$ 9, Clang $\geq$ 10, or MSVC 2019+
CMake	3.14 (Optional)	Required only if using Method B (CMake build system)

⚡ Method A — Direct g++ (Fastest & Recommended)

This method is highly recommended for MSYS2, MinGW (Windows), Linux, or macOS terminal environments. It compiles the entire project into a single binary with one command.

Step 1: Open Terminal in Project Root

Ensure your terminal is currently located in the root directory: `C:\HACKCPP`.

Step 2: Compile

Run the following command to compile the source code with maximum optimizations (`-O3`):

```
g++ -O3 -std=c++17 -Wall -Wextra -Wpedantic -DCONSOLIDATED_BUILD src/main.cpp
src/tests.cpp src/Sensor.cpp src/Alert.cpp src/Logger.cpp src/Dashboard.cpp
src/ThreadManager.cpp src/ServiceBus.cpp src/CanBus.cpp src/DtcManager.cpp
src/EcuMonitor.cpp src/OtaSimulator.cpp src/EdrRecorder.cpp -Iinclude -o
SmartVehicleMonitor.exe -pthread
```

Step 3: Run

Once compiled, you can execute the generated binary.

On Windows:

```
.\SmartVehicleMonitor.exe
```

On Linux / macOS:

```
./SmartVehicleMonitor.exe
```

[!WARNING] **Always run the executable from the project root folder** (e.g., `C:\HACKCPP`). The application relies on relative paths to locate the `data/` directory for configuration and the `logs/` directory for writing logs.

🔧 Method B — CMake (Cross-Platform / IDE Standard)

If you prefer using a standard build system or are integrating with an IDE like VS Code or CLion, use CMake.

Step 1: Create a Build Directory

From the root of the project (C : \HACKCPP), create and navigate to a new build folder:

```
mkdir build
cd build
```

Step 2: Configure the Project

Generate the build files using CMake:

```
cmake ..
```

Step 3: Compile

Build the executable in Release mode:

```
cmake --build . --config Release
```

Step 4: Run

Navigate back to the project root before running, or execute it directly from root:

On Windows:

```
cd ..
.\build\Release\SmartVehicleMonitor.exe
```

(Note: Depending on your CMake generator, the executable might just be in . \build\SmartVehicleMonitor.exe)

On Linux / macOS:

```
cd ..
./build/SmartVehicleMonitor
```

📄 Method C — Visual Studio (Windows Developer Command Prompt)

If you are strictly using Microsoft Visual Studio, you can generate the solution files and build directly from the command line.

Open the **Developer Command Prompt for VS**, navigate to C : \HACKCPP, and run:

```
cmake -B build -G "Visual Studio 17 2022"
cmake --build build --config Release
.\build\Release\SmartVehicleMonitor.exe
```

□ Startup & Execution Modes

Upon successfully running the executable, you will be greeted with a startup menu:

```
=====
|                               |
|          STARTUP SELECTION   |
|                               |
|-----|
| [1] Run all 10 automated test cases |
| [2] Select and run a specific test case |
| [3] Skip tests and start Dashboard directly |
|-----|
Enter option [1-3]:
```

- **Option 1:** Validates the entire system architecture by running the JSON-driven test suite. Results are written to `logs/test_report.csv`.
- **Option 3:** Directly launches the live ANSI-rendered telemetry dashboard.

Graceful Shutdown

While the live dashboard is running, simply press **ENTER** at any time. This signals the atomic running flag, gracefully joining all background threads and flushing all queues to disk before exiting.

Independent CLI Test Mode

You can also run dynamic free-text NLP tests directly from the command line without entering the main dashboard:

```
.\SmartVehicleMonitor.exe -testmode -user "Engine temperature is 115C, speed is 90kmh, door is open"
```